

서술형 1 번. 기능별 최적의 데이터베이스 설계

‘투빅스타일’ 서비스는 이미지 기반 추천, 사용자 활동 기록, 주문/결제, 로그 분석 등 서로 다른 성격의 데이터를 동시에 다루므로 하나의 데이터베이스만으로 모든 기능을 효율적으로 처리하기 어렵다. 따라서 기능별 요구사항에 맞추어 여러 종류의 DB 를 조합하는 방식이 적절하다.

1) 결제 및 주문 관리 시스템 — RDB 활용

결제, 주문, 환불, 배송 상태와 같은 데이터는 정확성과 일관성이 가장 중요함. 예를 들어 결제는 되었는데 주문 내역은 저장되지 않거나, 환불 기록은 있는데 실제 결제 상태는 반영되지 않는 상황이 발생하면 안된다. 따라서 트랜잭션의 원자성, 일관성, 격리성, 지속성을 보장하는 ACID 특성이 필요하며, 이를 가장 안정적으로 지원하는 관계형 데이터베이스(RDB)가 적합하다. 또한 주문, 사용자, 상품, 결제 테이블 간 관계를 명확하게 설계할 수 있어 데이터 무결성 유지에도 유리하다.

2) 스타일 이미지 유사도 검색 — Vector DB 활용

투빅스타일의 핵심 기능은 사용자가 업로드한 옷 사진과 유사한 상품을 빠르게 찾아 추천하는 것이다. 이를 위해 이미지에서 추출한 임베딩 벡터를 저장하고, 새로운 이미지와 가장 가까운 벡터를 실시간으로 검색해야 한다. 이러한 기능은 일반적인 RDB 보다 벡터 유사도 검색에 최적화된 Vector DB가 훨씬 적합하다.

Vector DB 를 사용하면 코사인 유사도나 최근접 이웃 탐색 등을 통해 비슷한 스타일의 상품을 빠르게 찾을 수 있으므로, 추천 속도와 품질을 동시에 높일 수 있다.

3) 사용자 활동 로그 및 클릭 기록 저장 — NoSQL 활용

사용자의 클릭, 조회, 찜, 검색어, 체류 시간 같은 행동 로그는 매우 많은 양이 빠르게 쌓이며 구조도 자주 바뀔 수 있다. 예를 들어 오늘은 “찜 여부”만 저장하다가 나중에는 “스크롤 깊이”, “추천 노출 위치” 같은 컬럼을 추가해야 할 수 있다. 이런 비정형·반정형 데이터는 스키마가 엄격한 RDB 보다 유연한 NoSQL이 적합하다.

NoSQL 을 사용하면 대용량 로그 데이터를 빠르게 저장할 수 있고, 서비스 확장 시 수평 확장도 쉬워 운영에 유리하다.

4) 추천 성능 분석 및 운영 리포트 — 데이터 웨어하우스 활용

운영팀과 데이터팀은 “어떤 추천 모델이 클릭률이 높았는지”, “어떤 스타일 카테고리에서 구매 전환이 잘 일어났는지”, “시간대별 추천 성능이 어떤지” 등을 주기적으로 분석해야 한다. 이런 분석 업무는 실시간 서비스 DB 에 직접 부담을 주는 것보다, 별도의 데이터 웨어하우스에 로그와 이벤트 데이터를 적재한 뒤 집계·분석하는 방식이 효율적이다.

데이터 웨어하우스를 활용하면 대시보드 구성, 모델 성능 비교, 사용자 행동 분석, A/B 테스트 결과 확인이 쉬워진다.

종합 설계안

정리하면, 주문·결제는 RDB, 이미지 유사 추천은 Vector DB, 사용자 로그 저장은 NoSQL, 장기 분석과 리포트는 데이터 웨어하우스로 분리하는 구조가 적절하다. 이처럼 기능별로 DB를 조합하면 각각의 데이터 특성에 맞는 최적의 성능과 안정성을 확보할 수 있다.

서술형 2 번. MLOps 도구를 활용한 'TO-BE' 인프라 구축

문제 A 는 개발 환경과 운영 환경의 불일치로 인해 배포 시 라이브러리 오류가 발생하는 문제이며, 문제 B 는 모델 성능 저하를 사전에 감지하지 못하는 문제이다. 이러한 문제를 해결하기 위해 재현 가능한 실행 환경, 자동화된 배포 파이프라인, 그리고 지속적인 성능 모니터링과 재학습 체계를 포함한 MLOps 인프라를 구축할 필요가 있다.

우선 문제 A 를 해결하기 위해 **Docker** 를 활용하여 모델 실행 환경을 컨테이너로 패키징한다. Python 버전, 주요 라이브러리, 모델 실행 코드 등을 하나의 이미지로 고정함으로써 개발 환경과 서버 환경을 동일하게 유지할 수 있으며, 이를 통해 “개발 환경에서는 정상 동작하지만 서버에서는 실패하는 문제”를 방지할 수 있다.

다음으로 **CI/CD 파이프라인**을 구축하여 코드 변경 시 자동으로 테스트와 배포가 이루어지도록 한다. 예를 들어 GitHub Actions 또는 Jenkins 를 활용하여 코드가 저장소에 반영되면 자동으로 단위 테스트를 수행하고, Docker 이미지를 빌드한 뒤 테스트 서버에 배포하고, 검증이 완료되면 운영 서버에 반영되도록 설계한다. 이를 통해 수동 배포 과정에서 발생할 수 있는 오류를 줄이고 안정적인 운영이 가능해진다.

또한 모델의 학습부터 배포까지 전 과정을 자동화하기 위해 **파이프라인 오케스트레이션 도구(Airflow 또는 Kubeflow)**를 활용한다. 데이터 수집, 전처리, 모델 학습, 성능 평가, 모델 등록, 배포까지의 과정을 하나의 워크플로우(DAG)로 구성하여 자동 실행되도록 한다. 이를 통해 사람이 개입하지 않아도 정해진 조건에 따라 전체 머신러닝 파이프라인이 일관되게 수행되며, 재현성과 관리 효율성이 크게 향상된다.

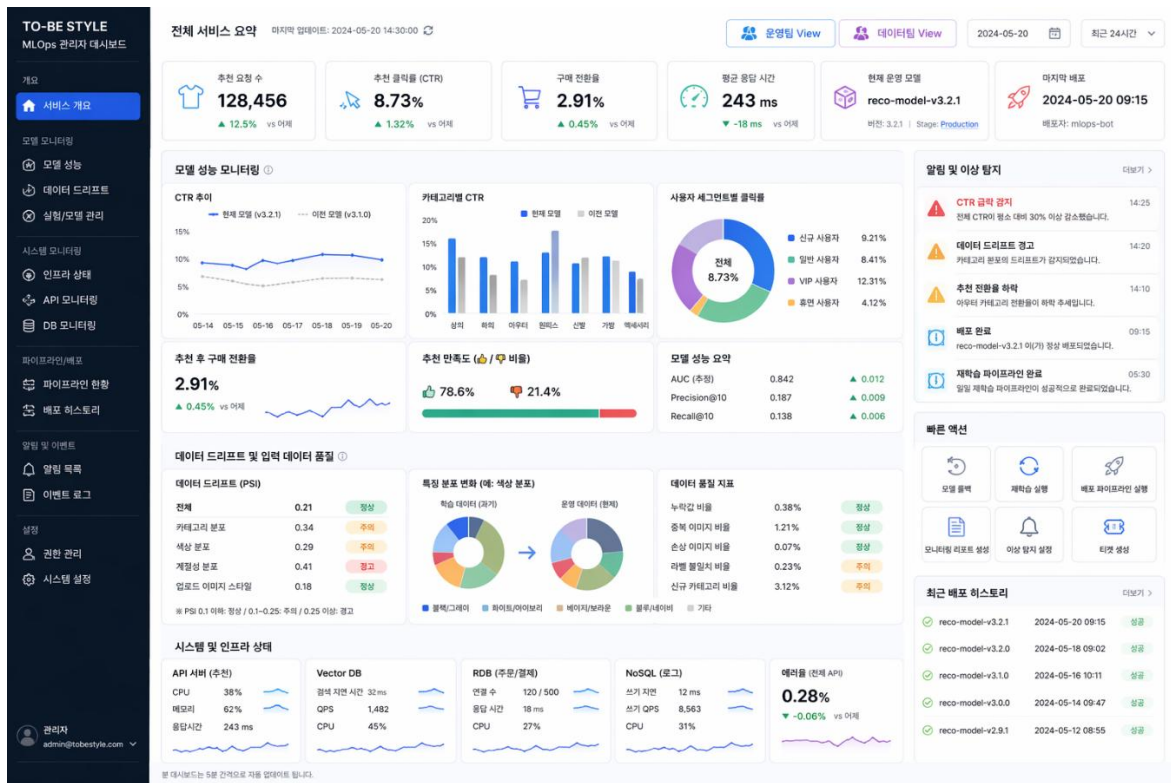
모델 관리 측면에서는 **MLflow** 를 활용하여 실험 결과, 하이퍼파라미터, 모델 성능 지표, 버전 정보를 체계적으로 기록한다. 이를 통해 어떤 모델이 언제, 어떤 데이터로 학습되었는지 추적할 수 있으며, 성능 저하 시 이전 모델로 쉽게 롤백할 수 있다.

문제 B 를 해결하기 위해서는 운영 중인 모델의 성능을 지속적으로 모니터링하는 체계가 필요하다. 이를 위해 추천 시스템의 주요 지표인 클릭률(CTR), 구매 전환율, 사용자 반응 데이터를 수집하고, **Prometheus** 와 **Grafana** 를 활용하여 실시간으로 시각화한다. 또한 **Evidently AI** 와 같은 도구를 활용하여 입력 데이터 분포 변화(데이터 드리프트)와 모델 성능 저하를 자동으로 감지할 수 있도록 한다.

나아가 성능 저하가 감지되었을 때 자동으로 모델을 개선할 수 있도록 **지속적 학습(Continuous Training, CT)** 체계를 구축한다. 예를 들어 최근 사용자 행동 로그와 최신 이미지 데이터를 기반으로 재학습 파이프라인을 자동 실행하고, 새롭게 학습된 모델의 성능이 기존 모델보다 우수한 경우에만 배포되도록 설계한다. 이 과정 역시 파이프라인 오케스트레이션과 연계되어 자동으로 수행되도록 구성할 수 있다.

서술형 3 번. 운영팀과 데이터팀이 함께 볼 수 있는 MLOps 관리자 대시보드 설계

운영팀과 데이터팀이 같은 화면을 보면서 현재 서비스 상태와 모델 상태를 함께 파악할 수 있는 방향으로 설계한다. 따라서 **서비스 운영 지표 + 모델 성능 지표 + 데이터 품질 지표**가 통합된 형태로 설계하는 것이 적절하다.



1) 대시보드 상단 — 전체 서비스 현황 요약

가장 위에는 오늘 기준 핵심 지표를 카드 형태로 배치한다.

- 오늘 추천 요청 수
- 추천 API 평균 응답 시간
- 추천 클릭률(CTR)
- 추천 후 구매 전환율

- 현재 운영 중인 모델 버전
- 마지막 배포 시각

이 영역은 운영팀이 서비스 이상 여부를 빠르게 파악할 수 있게 해주고, 데이터팀은 현재 어떤 모델이 실제로 서비스 중인지 즉시 확인할 수 있게 한다.

2) 모델 성능 모니터링 영역

중앙에는 모델 관련 지표를 배치한다.

- 일별 CTR 추이
- 카테고리별 추천 성능 비교
- 신상품/기존 상품 추천 성공률
- 사용자 세그먼트별 반응 차이
- 이전 모델 대비 현재 모델 성능 변화

이 영역은 데이터팀이 모델 품질을 검토하는 데 핵심적이다. 예를 들어 여성 아우터 카테고리에서만 클릭률이 하락하고 있다면 해당 카테고리 임베딩이나 학습 데이터 편향 문제를 의심할 수 있다.

3) 데이터 드리프트 및 입력 품질 영역

모델 성능이 떨어지는 원인이 데이터 변화 때문인지 확인할 수 있도록 아래 항목을 포함한다.

- 최근 업로드 이미지 분포 변화
- 색상, 카테고리, 계절성 피쳐 분포 변화
- 학습 데이터와 운영 데이터 간 유사도 차이
- 누락값/오류 이미지 비율
- 드리프트 경고 여부

이 화면은 데이터팀이 재학습 필요성을 판단하는 데 유용하며, 운영팀도 단순 서버 문제가 아닌 데이터 문제임을 구분할 수 있게 한다.

4) 시스템 운영 상태 영역

운영팀을 위해 인프라 상태도 함께 제공해야 한다.

- API 서버 CPU/메모리 사용량
- DB 응답 속도

- Vector DB 검색 지연 시간
- 에러율(5xx, timeout 등)
- 최근 배포 성공/실패 로그

이 영역을 통해 운영팀은 "추천 결과가 이상한 이유가 모델 문제인지, 서버 장애인지"를 빠르게 분리해서 판단할 수 있다.

5) 알림 및 액션 영역

오른쪽 패널에는 경고와 대응 기능을 배치한다.

- CTR 급락 알림
- 데이터 드리프트 발생 알림
- 최근 배포 실패 알림
- 이전 모델로 롤백 버튼
- 재학습 파이프라인 실행 버튼
- 장애 티켓 생성 버튼

이렇게 하면 대시보드가 단순 조회용이 아니라 실제 운영 대응 도구로 기능할 수 있다.

6) 기대 효과

이 대시보드를 운영하면 운영팀은 서버와 서비스 품질을 실시간으로 파악할 수 있고, 데이터팀은 모델의 성능 변화와 데이터 변화 원인을 빠르게 분석할 수 있다. 결국 두 팀이 같은 지표를 공유함으로써 "문제가 생긴 뒤 고객이 먼저 불만을 제기하는 방식"이 아니라, 내부적으로 먼저 이상을 감지하고 대응하는 선제적 운영이 가능해짐.